From C to Java: Evaluating LLMs on Cross-Language Graph Algorithm Synthesis

Simon Kraft

Student ID: 230171256
Email: kraft@unbc.ca

CPSC 482/682 — Data Structures II
University of Northern British Columbia

Date of Submission: April 3, 2026

Abstract

Large language models (LLMs) are increasingly used for code generation and the automation of software development, yet most evaluations focus on high-level programming languages such as Python. This study extends the work "Evaluating Efficiency and Novelty of LLM-Generated Code for Graph Analysis" by Nia, Dindoost, and Bader [9] by examining whether state-of-the-art LLMs are capable of generating efficient Java implementations of graph algorithms when provided only with the C language source files as context. Our benchmark comprises three LLMs (Anthropic Claude Opus 4.6, Google Gemini 3.1 Pro, and OpenAI GPT-5.3-Codex) and their code generation is evaluated on two different approaches similar to the previous work. The first optimization approach queries the LLMs to generate the most efficient routine for the problem of finding the number of triangles in a graph, while being given a series of human-written C implementations that solve this problem. The second algorithm-synthesis approach requires the LLMs to generate Java implementations for Triangle Counting, Graph Diameter, and Clique Number, where the models are solely provided with the benchmark suite in C and need to make assumptions about an equivalent benchmark suite in Java, without actually seeing it. The implementations are evaluated based on compilability, correctness, runtime efficiency, and memory footprint using synthetic RMAT benchmark graphs. Our results reveal significant differences between the evaluated models in the cross-language setting. While the optimization approach showed broad success across all models, the algorithm-synthesis approach was highly dominated by Google's Gemini 3.1 Pro model with 100% RTU implementations. Both other models struggled with API consistency and therefore performed poorly on the second approach. These findings highlight the difficulty of structural abstraction in LLM code generation and suggest that current efficiency metrics are highly sensitive to small-sample evaluation suites.

1 Introduction

Large language models (LLMs) have seen broad adoption across nearly every sector, assisting with tasks ranging from natural language understanding to complex reasoning and software development [5]. One of their most impactful applications is code generation, where they help programmers by automating development tasks and reducing human error [9]. While most existing benchmarks such as HumanEval [6] and MBPP [2] focus primarily on correctness and functionality, the performance of written code in terms of memory footprint and runtime efficiency is largely overlooked. However, in scientific computing and graph analysis small algorithmic optimizations can result in a reduction of runtime and energy consumption by orders of magnitude [9]. Barati Nia, Dindoost, and Bader [9] addressed this by benchmarking eight LLMs on their ability to generate efficient C implementations of graph algorithms, supplying each model with an existing C benchmark suite [3] to reduce the risk of memorized training data reproduction.

A natural extension is to evaluate LLMs when the target language differs from the provided context — a scenario of practical relevance as many legacy systems are written in lower-level languages such as C or COBOL [7], while new development targets higher-level languages such as Java. This cross-language setting is strictly harder than same-language generation: since no Java infrastructure is provided, each model must infer the Java API from the C source files alone and generate code that integrates into an imagined Java framework.

This work extends the evaluation of Barati Nia et al. [9] under this cross-language setting. Three state-of-the-art LLMs are prompted to generate Java implementations of graph algorithms while only C source files are provided as context. Two approaches are evaluated: an optimization approach for triangle counting, and an algorithm-synthesis approach covering triangle counting, diameter finding, and clique number, using RMAT benchmark graphs [4] in CSR format. Each implementation is evaluated on compilability, correctness, runtime, and memory usage, following the methodology of the original paper [9]. To evaluate the obtained implementations, the benchmark suite in C was translated to Java beforehand.

The remainder of this report is organized as follows. Section 2 describes the experimental setup and reports numerical results compared against the original paper [9]. Section 3 summarizes findings and suggests future directions.

2 Numerical Testing

2.1 Testing Environment

All experiments were conducted on an Apple MacBook Pro with an Apple M3 chip featuring 4 performance cores running at up to 4.05 GHz and 4 efficiency cores running at up to 2.75 GHz, with 16 GB of unified DRAM. The performance cores feature 192 KB of L1 instruction cache, 128 KB of L1 data cache, and a shared 16 MB L2 cache. The efficiency cores feature 128 KB of L1 instruction cache, 64 KB of L1 data cache, and a shared 4 MB L2 cache. Java implementations were compiled and executed using Java 21. Each algorithm is averaged over 10 runs to smooth out measurement noise, consistent with the methodology of the original paper [9]. Note that absolute runtime results are not comparable with the original study which used an AMD EPYC 7753 server at 2.45 GHz with 512 GB of RAM [9].

2.2 Employed Large Language Models

At the time of writing, three frontier large language models from the three most dominant AI providers were selected, each recognized for their strong reasoning and code generation capabilities.

Anthropic Claude Opus 4.6 (released February 5, 2026 [1]) is Anthropic's most capable model, designed for superior coding, complex reasoning, and agentic workflows with a 1M token context window. Compared to Claude Sonnet 4 used in the original paper, it achieves better results on Terminal-Bench 2.0, leading us to expect stronger implementations.

Google Gemini 3.1 Pro (released February 19, 2026 [8]) represents a substantial improvement over the Gemini 2.5 models tested in the original paper, achieving more than double the reasoning performance on competitive coding benchmarks, making it a compelling candidate to re-evaluate Google's standing.

OpenAI GPT-5.3-Codex (released February 5, 2026 [10]), part of OpenAI's dedicated coding model family, advances both frontier coding performance and reasoning capabilities. It was accessed via the GitHub Copilot web interface.

2.3 Prompt Design

The altered prompts are given in Figure 1 and Figure 2 in the Appendix. In order to achieve the cross-language setting, the prompts are adapted from [9] with two modifications: the target language is changed from C to Java, and the phrase *"Suppose this whole framework is already translated to Java"* is added to inform models that a Java infrastructure exists without providing it. This forces the models to infer the Java API from the C source alone. Each LLM was cold-started before each prompt [9].

2.4 Optimization Approach Results

Table 3 summarizes the results of the optimization approach on RMAT-16. In contrast to the original paper, where three out of eight models failed to produce correct triangle counts [9], all three models in our evaluation compiled without any manual modification and produced correct results. This finding suggests that providing the full triangle counting implementation as context is sufficient for reliable cross-language generation.

All three implementations substantially outperformed the brute-force baseline (27.07 s), drawing on techniques present in the provided `tc.c` source file. GPT-5.3-Codex achieved the fastest runtime (0.298 s, $\approx 91\times$ faster) by employing all three optimizations available in the C context, degree-based vertex sorting, hash-based intersection, and the Forward Triangle Counting algorithm, at the cost of 25.73 MB of peak memory. Claude Opus 4.6 achieved a nearly identical runtime (0.318 s) using only hash-based intersection and FTC, without sorting. Gemini 3.1 Pro was $3.5\times$ slower at 1.032 s but required zero additional memory, relying solely on FTC with a merge-path intersection over the already-sorted adjacency lists. This strategy trades runtime for memory efficiency. However, whether each model genuinely reasoned from the C source or reproduced patterns from training data remains an inherent limitation of this evaluation setup [9].

2.5 Algorithm-Synthesis Approach Results

1) Read-To-Use Capability: Table 1 reports the Ready-To-Use (RTU) percentage for each model across the three graph algorithm tasks. Similar to Nia, Dindoost, and Bader [9], the implementation of an algorithm is defined as Ready-To-Use (RTU) if the generated code meets all of the following criteria:

- **Compilability:** The code must compile successfully without any manual modifications.
- **Correctness:** The code must produce correct outputs on all test cases.
- **Timeliness:** The implementation must run within an acceptable runtime threshold to ensure practical usability.

Table 1: RTU Percentage per Model.

Model	RTU Percentage
Anthropic Claude Opus 4.6	33 %
Google Gemini 3.1 Pro	**100 %**
OpenAI GPT-5.3-Codex	0 %

Gemini 3.1 Pro achieved the highest RTU rate of 100%, successfully generating compilable, correct, and timely implementations for all three algorithms. Claude Opus 4.6 achieved 33%, with only triangle counting passing all three criteria. Its diameter implementation required a manual compilability fix, and its clique implementation produced incorrect results. GPT-5.3-Codex failed to achieve any RTU implementations, with compilability failures in triangle counting and clique number, and a timeliness failure in diameter finding.

Compared to the original paper, where RTU rates ranged from 17% to 83% across eight C implementations, our cross-language setting shows greater variance. Gemini 3.1 Pro surpasses even the best result from the original paper (Claude Sonnet 4 Extended, 83%), while GPT-5.3-Codex scores 0%. This suggests that cross-language generation amplifies differences between models rather than uniformly degrading performance.

2) Efficiency Trade-Offs: Table 2 shows the combined efficiency rates for each model, computed as the sum of $1/(t \cdot m)$ across all RTU algorithm implementations, where t is the relative runtime and m is the relative peak memory usage, similar to [9].

Table 2: Efficiency Rates

Model	Efficiency Rate
Anthropic Claude Opus 4.6	0.15
Google Gemini 3.1 Pro	**2.63**
OpenAI GPT-5.3-Codex	0.00

The efficiency rates reflect the RTU results directly. Gemini 3.1 Pro achieves the highest total efficiency rate of 2.63, contributing rates across all three algorithms. Claude Opus 4.6

contributes only through its RTU triangle counting implementation, yielding a rate of 0.15, while GPT-5.3-Codex contributes nothing. Compared to the original paper's best result of 3.11 (Claude Sonnet 4 Extended), Gemini 3.1 Pro's 2.63 is competitive, suggesting that the newer Gemini model has significantly closed the gap with Anthropic's models observed in the original study. However, the low rates of the other models reflect a key limitation of the cross-language setting, where small compilability failures, such as using accessor methods instead of direct attribute access, caused by incorrect API assumptions, disproportionately penalizes the LLMs. Furthermore, with only three algorithms evaluated, a single failure has an outsized effect on the total score. Evaluating a broader algorithm suite would likely produce more differentiated and balanced results.

3 Summary and Conclusions

This project evaluated three LLMs on their ability to generate efficient Java graph algorithm implementations when provided with C source files as context. In the optimization approach, each model was able to generate a correct solution, with GPT-5.3-Codex and Claude Opus 4.6 achieving approximately 91× speedup over the naive baseline. In contrast, Gemini 3.1 Pro implemented an in-place algorithm that was slower than its competitors but still outperformed the baseline with a speedup of roughly 26×. Despite these strong results, Gemini was the only model to continue this strong trend in the algorithm synthesis with 100% RTU implementations for Triangle Counting, Graph Diameter, and Clique Number. While the Codex model from OpenAI failed to produce RTU Java code, due to lack of compilability and timeliness, Claude Opus was able to generate a correct Triangle Counting routine that was even faster than the one from Gemini.

The central challenge in cross-language code generation lies in the API assumptions. Unlike the original paper, where models could directly reuse field names and function signatures from the provided C source, our setting required each model to abstract away the source language and infer a plausible Java API independently. Even state-of-the-art models failed at this. Furthermore, it is difficult to assess whether models genuinely reasoned from the provided C context or simply reproduced Java graph algorithm patterns seen during training. This is an inherent limitation of any LLM evaluation without training data transparency. Furthermore, the introduced efficiency metric itself amplified small differences in this setting. With only three algorithms evaluated, a single compilability failure has an outsized effect on the total score, and the RTU definition does not distinguish between a near-miss requiring one-line fix and a fundamentally incorrect implementation. A broader algorithm suite and a partial-credit RTU metric would produce more differentiated and informative results.

Future research could expand the evaluation suite to include edge and vertex connectivity, to offer a more nuanced assessment of model performance in the cross-language setting [9]. Furthermore, investigating the translation across more divergent languages, such as C and Rust or COBOL and Python, could reveal whether the difficulty of the API integration scales with conceptual difference between programming languages. Finally, implementing iterative feedback loops, where models see compiler diagnostics and are allowed self-correction, provides a promising direction to overcome the pitfalls of the current efficiency rate metric.

Acknowledgements

I hereby declare that I have made limited use of generative artificial intelligence during the preparation of this project. Specifically, Claude Sonnet 4.6 and Gemini 2.5 Flash were used to aid the translation of the C infrastructure files into Java (`Graph.java`, `BFS.java`, `Queue.java`, `Main.java`). Furthermore, they were used for guidance on overall project setup and Java compilation. All experimental results, analysis, and the written content are my own work.

References

[1] Anthropic. *Introducing Claude Opus 4.6*. https://www.anthropic.com/news/claude-opus-4-6. Accessed: March 2026. Feb. 2026 (cit. on p. 3).

[2] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton. *Program Synthesis with Large Language Models*. 2021. arXiv: 2108.07732 [cs.PL]. URL: https://arxiv.org/abs/2108.07732 (cit. on p. 2).

[3] D. A. Bader. "Fast Triangle Counting". In: *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. 2023, pp. 1–6 (cit. on p. 2).

[4] D. Chakrabarti, Y. Zhan, and C. Faloutsos. "R-MAT: A Recursive Model for Graph Mining". In: *Proceedings of the 2004 SIAM International Conference on Data Mining*. 2004, pp. 442–446 (cit. on p. 2).

[5] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, et al. "A Survey on Evaluation of Large Language Models". In: *ACM Transactions on Intelligent Systems and Technology* 15.3 (2024), pp. 1–45 (cit. on p. 2).

[6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: https://arxiv.org/abs/2107.03374 (cit. on p. 2).

[7] L. Erlikh. "Leveraging legacy system dollars for e-business". In: *IT Professional* 2.3 (2000), pp. 17–23 (cit. on p. 2).

[8] Google DeepMind. *Gemini 3.1 Pro*. https://deepmind.google/models/gemini/pro/. Accessed: March 2026. Feb. 2026 (cit. on p. 3).

[9] A. B. Nia, M. Dindoost, and D. A. Bader. "Evaluating Efficiency and Novelty of LLM-Generated Code for Graph Analysis". In: *2025 IEEE High Performance Extreme Computing Conference (HPEC)*. 2025, pp. 1–7 (cit. on pp. 1–5, 8).

[10] OpenAI. *Introducing GPT-5.3-Codex*. https://openai.com/index/introducing-gpt-5-3-codex/. Accessed: March 2026. Feb. 2026 (cit. on p. 3).

Appendix

Optimization Approach

```
The attached C code implements triangle counting in a graph given in CSR format.
Suppose this whole framework is already translated to Java. Write a Java method
tc_fast() with the API below that is the fastest sequential triangle counting
code you can generate:

  public static long tc_fast(Graph graph)

Attached Files: (bfs.c, graph.c, main.c, queue.c, tc.c)
```

Figure 1: Prompt structure for the optimization approach, adapted from [9] Fig. 1.

Algorithm-Synthesis Approach

```
The attached C files implement a graph in CSR format. Suppose this whole
framework is already translated to Java. Write a Java method fast() with the API
below that is the fastest sequential implementation for finding the {ALGORITHM}
of a graph that integrates into the already translated Java framework:

  public static long fast(Graph graph)

Attached Files: (bfs.c, graph.c, main.c, queue.c)
```

Figure 2: Prompt structure for the algorithm-synthesis approach, adapted from [9] Fig. 2. {`ALGORITHM`} is replaced by "number of triangles", "diameter", and "clique number".

Table 3: Comparative Overview of the LLMs — Optimization Approach. Compilable: compiled without modifications. Correct: produced the correct triangle count. Runtime and Memory measured on RMAT-16[†]. Sort: sorted vertices by degree at beginning. Hash: used hash tables for intersection operations. FTC: used the forward triangle counting algorithm. BFS: employed breadth-first search.

Model	Compilable	Correct	# Triangles	Runtime (s)	Mem (MB)	Sort	Hash	FTC	BFS
Baseline (brute-force)	✓	✓	21,236,020	27.0682	0.12	✗	✗	✗	✗
Claude Opus 4.6	✓	✓	21,236,020	0.3182	11.00	✗	✓	✓	✗
Gemini 3.1 Pro	✓	✓	21,236,020	1.0315	**0.00**	✗	✗	✓	✗
GPT-5.3-Codex	✓	✓	21,236,020	**0.2979**	25.73	✓	✓	✓	✗

[†]Unlike the original paper, which reports relative memory usage normalized to ChatGPT o3, we report absolute peak memory consumption in MB measured via JVM heap delta, as no equivalent baseline model exists in our setup.